CS 645

Security and Privacy in Computer Systems

Lectures 8 & 9: Web Security I

- HTTP and HTML
- Public Key Infrastructure (PKI)
- Secure Sockets Layer (SSL)
- Case Study: The “Heartbleed” attack

HTTP and HTML

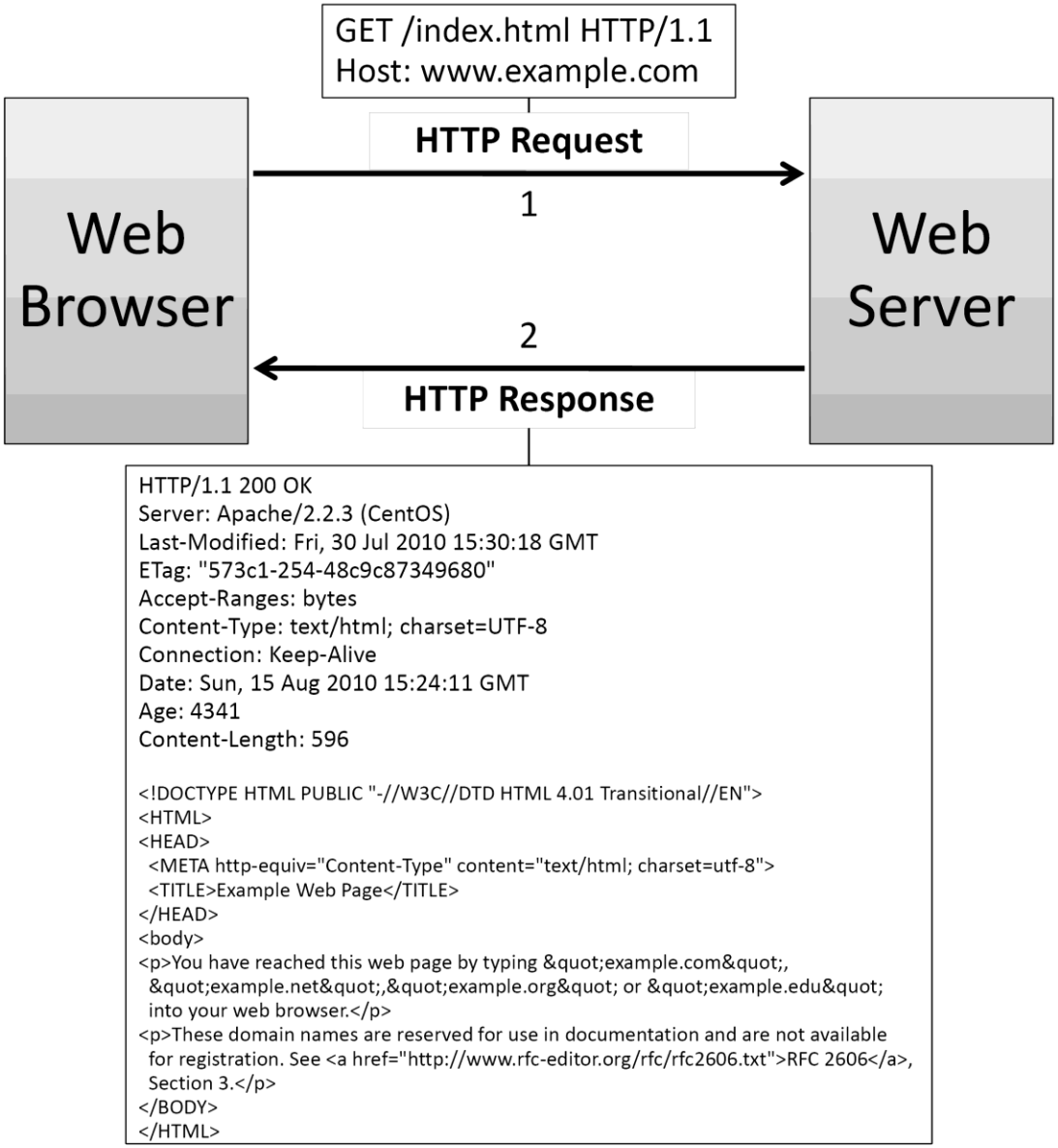
The World Wide Web (WWW)

- WWW is pervasive in our daily lives
- Websites use domain names
  - Conversion between domain names and IP addresses is done using the Domain Name System (DNS)
- Websites are identified using URLs (uniform resource locator)
  - http://www.example.com/directory/file.html
    - http = protocol
    - www.example.com = domain name of the web server
    - directory = the folder storing the website
    - file.html = a file for a webpage, using HTML (hypertext markup language)

HTTP

- To access a URL, a client's web browser makes a TCP connection to a specified port on the webserver (by default, HTTP uses port 80)
- HTTP request to retrieve the resource indicated by a URL
  - GET / POST
  - Headers section
  - Body section (contains form variables for POST)
- HTTP response
  - Headers section
    - Type and version number of software used on webserver; this information can be useful to attackers, in order to exploit specific vulnerabilities
  - Body section

HTTP request / response example



HTML

- Hypertext Markup Language (HTML)
  - Describes the content and formatting of Web pages
  - Rendered within browser window
- HTML features
  - Static document description language
  - Supports linking to other pages and embedding images by reference
  - User input sent to server via forms
- HTML extensions
  - Additional media content (e.g., PDF, video) supported through plugins
  - Embedding programs in supported languages (e.g., JavaScript, Java) provides dynamic content that interacts with the user, modifies the browser user interface, and can access the client computer environment

HTML – common tags

- Text formatting
  - Example: text , bold
- Itemized lists
  - Example:

```
<ul>
    <li> first-item </li>
    <li> second-item</li>
</ul>
```

- Hyperlinks
  - Example: <a href=www.ibm.com>IBM</a>
- Scripting code
  - Example: <script>code</script>
- Embedded images
  - Example: <img src="image-URL">

HTML forms

- Allow users to provide input to a website
  - Input consists of name-value variables, which can be then interpreted by the server
  - Form data can be submitted using GET or POST
- The GET method: name-value pairs are encoded directly into the URL, separated by &
  - http://www.example.com/form.php?first=Roberto&last=Tamassia
- The POST method: form input variables are included in the HTTP request's body section

HTML form example

```
<html>
    <title>Registration</title>
  <body>
    <h1>Registration</h1>
    <h2>Please enter your name and email address.</h2>
    <form method="GET" action="http://securitybook.net/register.php">
      First: <input type="text" name="first">
      Last: <input type="text" name="last">
      Email: <input type="text" name="email">
      <input type="submit" value="Submit">
    </form>
    <p><b>Thanks!</b></p>
  </body>
</html>
```

Code Fragment 7.1: HTML code for a simple registration page that contains a form with three variables to be submitted using the GET method.

HTML form example

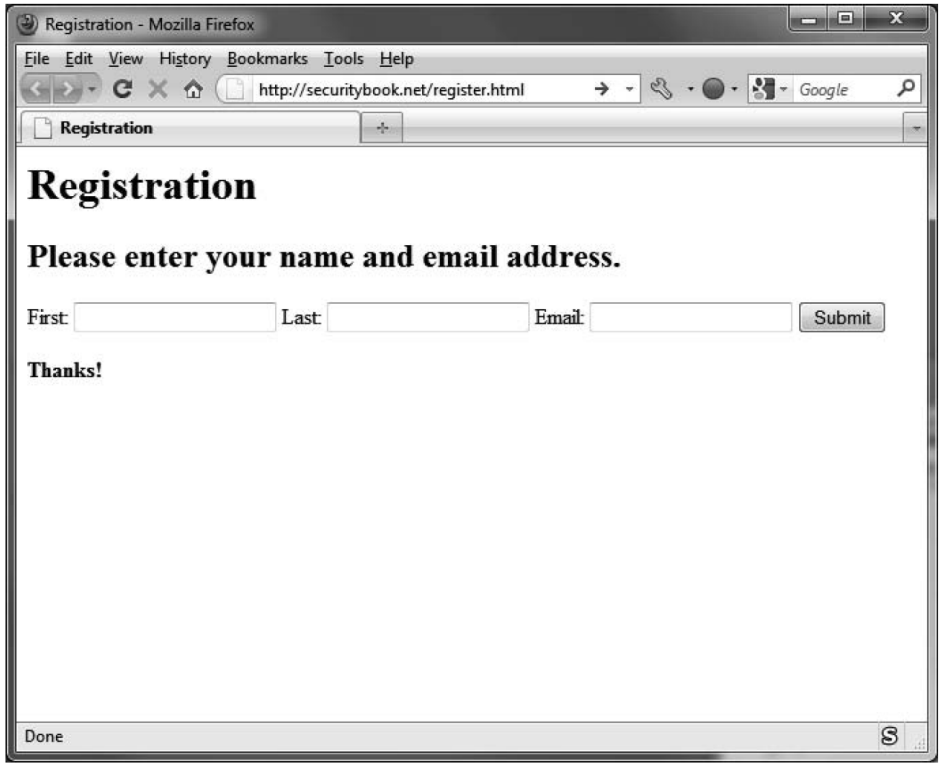


Figure 7.3: Web page from Code Fragment 7.1 displayed by the Mozilla Firefox web browser.

- When user clicks on the submit button, browser is redirected to:

http://securitybook.net/register.php?first=Roberto&last=Tamassia&email=rt%40securitybook.net

HTML form

- HTTP requests and responses are delivered over TCP over port 80
  - Data is sent ***in the clear***
  - Vulnerable to a man-in-the-middle attack, in which the attacker simply listens to the communication

- Q: How to secure communication?
- A: By using HTTPS (hypertext transfer protocol over secure socket layer)
  - HTTPS is identical with HTTP syntactically
  - Incorporates an additional layer of security, SSL/TLS (secure socket layer/transport layer security)

Public Key Infrastructure (PKI)

Public Key cryptographic primitives

$(Pub_A, Priv_A)$

Alice

M, Sig

Bob

?

generate signature

$Sig = S_{PrivA}(M)$

SIGNATURE

$V_{PubA}(M, Sig) =$ "valid"

$(Pub_B, Priv_B)$

Alice

C

Bob

generate ciphertext

$C = E_{PubB}(M)$

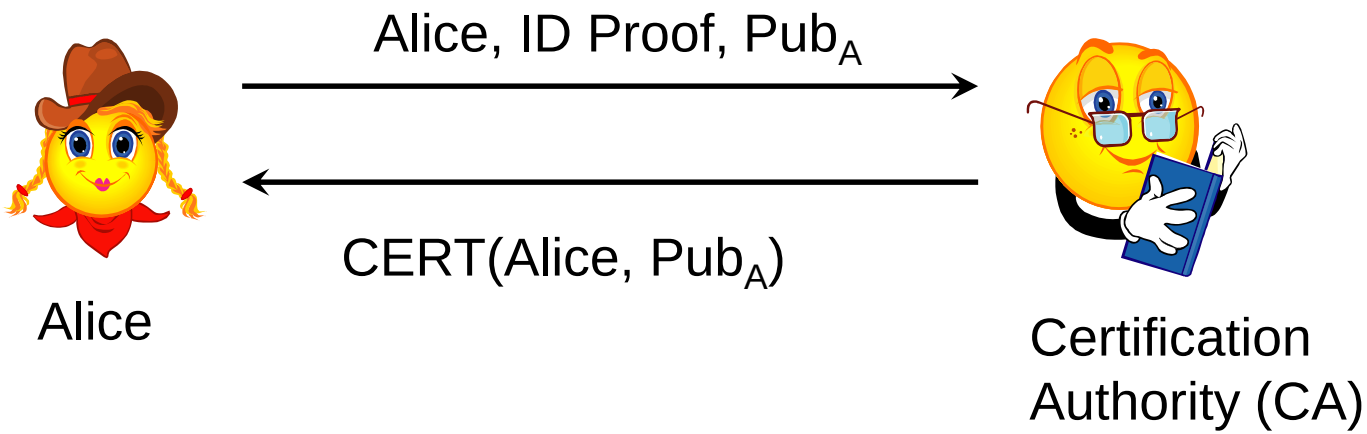
ENCRYPTION

retrieve plaintext

$M = D_{PrivB}(C)$

How to ensure the authenticity of the other's party public key?

Public Key Infrastructure (PKI)



- CERT(Alice, Pub_A) is Alice's digital certificate, which binds Alice's identity to her public key
  - The certificate contains a signature by the CA (using the CA's private key over the quantity (Alice, Pub_A)
- Anyone can verify authenticity of $CERT_A$ by using the CA's public key
- The CA's public key is readily available in a ***root certificate***
  - Included in the browser, published online, or in a newspaper, or on a CD etc.
  - The root certificate is a ***self-signed certificate*** (signed with the private key corresponding to the actual public key contained in the certificate)

PKI

- A public key certificate contains several fields:
  - The identity of the public key's owner
  - The public key
  - Serial number
  - Expiration date
  - Other useful fields
  - A signature over all these fields (computed with the CA's private key)

Public Key Infrastructure (PKI)

- When Alice needs Bob's public key, she retrieves Bob's certificate: CERT(Bob, Pub_B)
  - Alice has the authentic public key of the CA, so she can verify the authenticity of Bob's certficate
  - This validates the authenticity of Bob's public key, Pub_B, which is contained inside Bob's certificate

- PKI = the entire infrastructure needed to support public key cryptography
  - Includes organizations (CAs), principals, and their interactions

Problems with PKI?

- Public key certificates have expiration dates
  - Short term (a few months), or longer term (a few years)
- What happens when a certificate needs to be revoked before the expiration date?
  - A company goes out of business
  - A web domain changes ownership
  - An employee changes affiliation (e.g., leaves a company)
  - The private key gets compromised (!)
  - The private key is lost (!)

Problems with PKI?

- Certificate Revocation Lists (CRL) are lists of certificates (serial numbers more precisely) that have been revoked
  - Published periodically by the CA that issued the corresponding certificate
- Best practices require to always check if a certificate has not been revoked before expiration
  - This means, that in order to PKI effectively, one needs to always check the list of current CRLs
  - Thus, there is a need for an entity which is always available ("on-line") and provides CRLs
  - This requirement of "on-line validation" negates one of the original major advantages of PKI over symmetric key cryptography
  - The need for verifying CRL before certification validation also raises the possibility of DoS attacks against the PKI

Public key cryptography

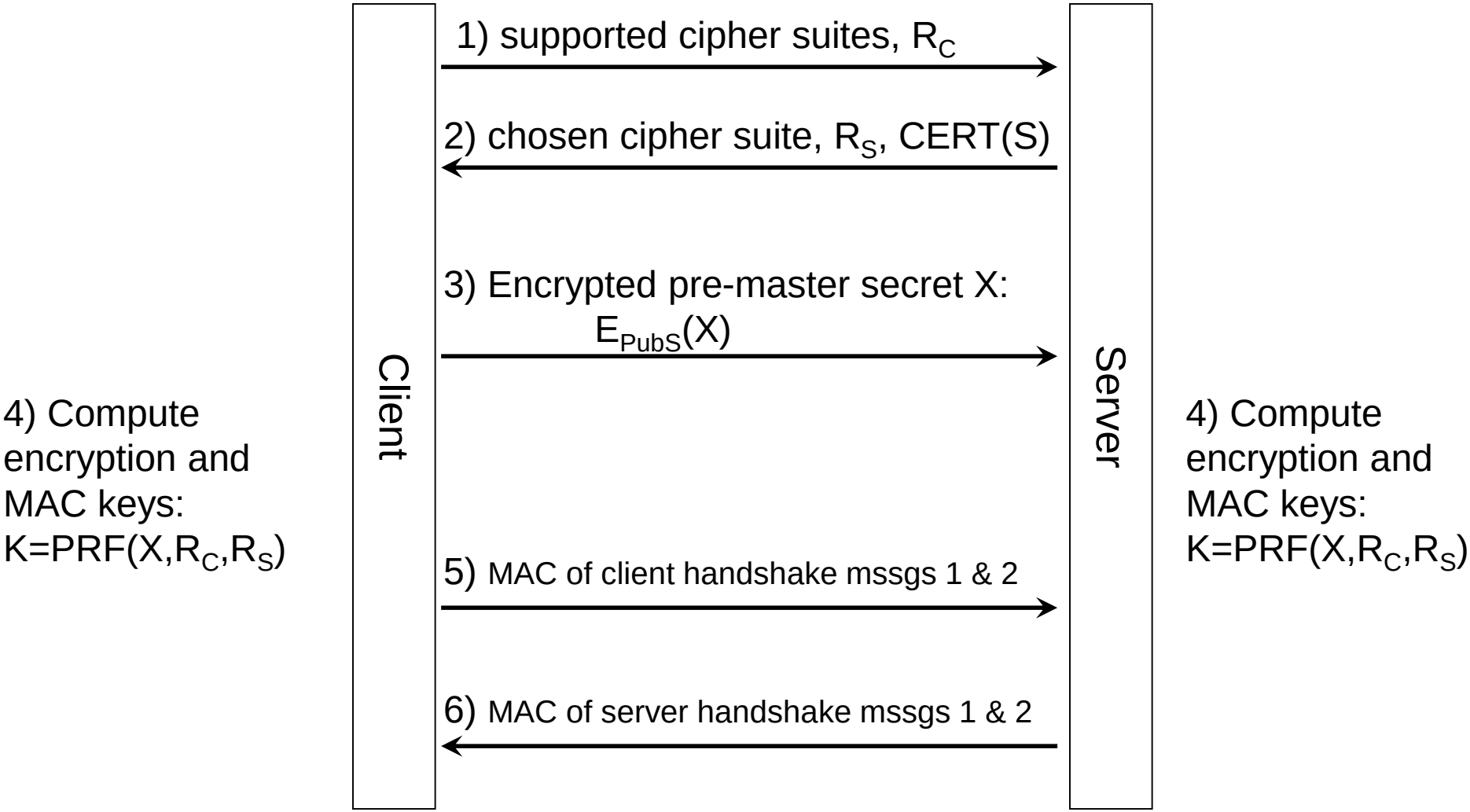
- Advantages over symmetric key crypto
  - Key management
    - Key establishment: does not require secure channel to transmit secret keys
    - Key distribution: does not require $O(n^2)$ keys to be managed to communicate with n entities
    - Can provide non-repudiation
- Disadvantages over symmetric key crypto
  - Slower (orders of magnitude)
- Is not meant to completely replace symmetric key cryptography, but to supplement it
  - $E_{Pub}(K)$, $E_K(M)$

Secure Sockets Layer (SSL)

Secure Sockets Layer (SSL)

- Goals for secure communication between a client and a server (browser and web server)
  - Provide confidentiality
  - Provide (mutual) authentication
- Communication is divided into two phases
  - Handshake
    - client and server agree on what set of cryptographic algorithms to use
    - authenticates the server
    - establishes keys which will be used for the data transfer
  - Secure data transfer
    - Data is transferred (authenticated and encrypted using the keys established during handshake)

Basic handshake protocol

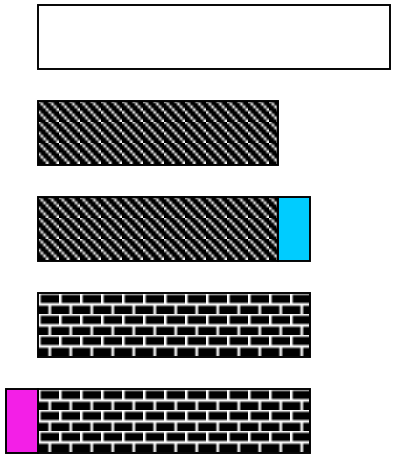


Basic handshake protocol

- Client uses https:// instead of http:// (this indicates to server to use port 443 instead of 80)
- X is a random value chosen by the client
- Only the server authenticates itself
  - Client authentication is rarely achieved through SSL
- Messages 5) and 6) protect tampering of the (unprotected) messages 1) and 2)
- The derived key material K includes <u>different</u> keys for encryption and authentication
  - $PRF(X, R_C, R_S) = (K_{S1}, K_{S2}, K_{C1}, K_{C2})$
  - Server encryption key K_{S1}, server MAC key K_{S2}
  - Client encryption key K_{C1}, client MAC key K_{C2}
- Msg 5: MAC_{KC2}(“CLNT”, msg1, msg2)
- Msg 6: MAC_{KS2}(“SRVR”, msg1, msg2)

SSL data transfer

- Data is broken into *records*
- Each record is:
  - Compressed
  - MAC-ed
  - Encrypted
  - Pre-pended with an SSL header



Early versions of SSL

- The presented protocol is SSL v3
- SSL v2 was flawed:
  - MITM attack (messages 5) and 6) were not present; attacker could tamper with messages 1) and 2) and cause a *downgrade* attack to a weak ciphersuite
  - Uses identical keys for both encryption and authentication
- SSL v2 is disabled by default in modern browsers

Certificates in SSL – root certificates

Keychain Access

Click to unlock the System Roots keychain.

Keychains
- login
- Micr...ertificates
- Micr...ertificates
- System
- System Roots

Category
- All Items
- Passwords
- Secure Notes
- My Certificates
- Keys
- Certificates

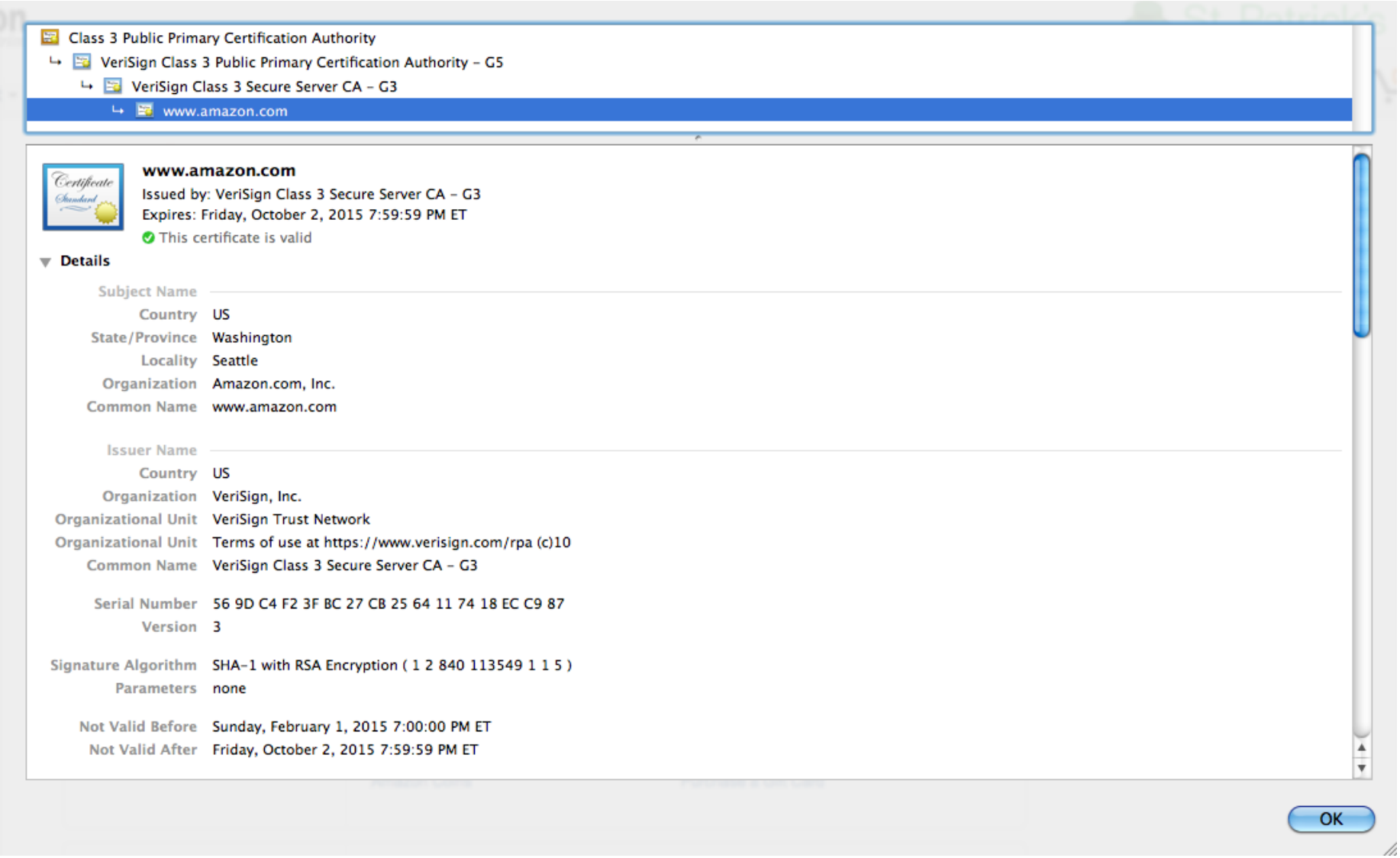
A-Trust-nQual-03
Root certificate authority
Expires: Monday, August 17, 2015 6:00:00 PM ET
This certificate is valid

Name	Kind	Date Modified	Expires	Keychain
A-Trust-nQual-03	certificate	--	Aug 17, 2015 6:00:00 PM	System Roots
AAA Certificate Services	certificate	--	Dec 31, 2028 6:59:59 PM	System Roots
AC Raíz Certicámara S.A.	certificate	--	Apr 2, 2030 5:42:02 PM	System Roots
Actalis Authentication Root CA	certificate	--	Sep 22, 2030 7:22:02 AM	System Roots
AddTrust Class 1 CA Root	certificate	--	May 30, 2020 6:38:31 AM	System Roots
AddTrust External CA Root	certificate	--	May 30, 2020 6:48:38 AM	System Roots
AddTrust Public CA Root	certificate	--	May 30, 2020 6:41:50 AM	System Roots
AddTrust Qualified CA Root	certificate	--	May 30, 2020 6:44:50 AM	System Roots
Admin-Root-CA	certificate	--	Nov 10, 2021 2:51:07 AM	System Roots
AdminCA-CD-T01	certificate	--	Jan 25, 2016 7:36:19 AM	System Roots
AffirmTrust Commercial	certificate	--	Dec 31, 2030 9:06:06 AM	System Roots
AffirmTrust Networking	certificate	--	Dec 31, 2030 9:08:24 AM	System Roots
AffirmTrust Premium	certificate	--	Dec 31, 2040 9:10:36 AM	System Roots
AffirmTrust Premium ECC	certificate	--	Dec 31, 2040 9:20:24 AM	System Roots
America Onli...ation Authority 1	certificate	--	Nov 19, 2037 3:43:00 PM	System Roots
America Onli...ation Authority 2	certificate	--	Sep 29, 2037 10:08:00 AM	System Roots
AOL Time W...cation Authority 1	certificate	--	Nov 20, 2037 10:03:00 AM	System Roots
AOL Time W...cation Authority 2	certificate	--	Sep 28, 2037 7:43:00 PM	System Roots
Apple Root CA	certificate	--	Feb 9, 2035 4:40:36 PM	System Roots
Apple Root Certificate Authority	certificate	--	Feb 9, 2025 7:18:14 PM	System Roots
Application CA G2	certificate	--	Mar 31, 2016 10:59:59 AM	System Roots
ApplicationCA	certificate	--	Dec 12, 2017 10:00:00 AM	System Roots
Autoridad de...l CIF A62634068	certificate	--	Dec 31, 2030 3:38:15 AM	System Roots
Autoridad de...tado Venezolano	certificate	--	Dec 17, 2030 6:59:59 PM	System Roots
Baltimore CyberTrust Root	certificate	--	May 12, 2025 7:59:00 PM	System Roots
Belgium Root CA2	certificate	--	Dec 15, 2021 3:00:00 AM	System Roots
Buypass Class 2 CA 1	certificate	--	Oct 13, 2016 6:25:09 AM	System Roots
Buypass Class 2 Root CA	certificate	--	Oct 26, 2040 4:38:03 AM	System Roots

Copy

200 items

Certificates in SSL – website certificates



Certificates in SSL – website certificates

Class 3 Public Primary Certification Authority
↳ VeriSign Class 3 Public Primary Certification Authority – G5
↳ VeriSign Class 3 Secure Server CA – G3
↳ www.amazon.com

Common Name	VeriSign Class 3 Secure Server CA – G3
Serial Number	56 9D C4 F2 3F BC 27 CB 25 64 11 74 18 EC C9 87
Version	3
Signature Algorithm	SHA-1 with RSA Encryption (1 2 840 113549 1 1 5)
Parameters	none
Not Valid Before	Sunday, February 1, 2015 7:00:00 PM ET
Not Valid After	Friday, October 2, 2015 7:59:59 PM ET
Public Key Info	
Algorithm	RSA Encryption (1 2 840 113549 1 1 1)
Parameters	none
Public Key	256 bytes : B9 33 ED 32 A7 8B 59 CD 63 72 62 34 01 C5 8C D7 AE 74 C7 E0 7E 25 B2 72 6C 6F 52 C8 21 B4 BA 36 FB 59 9F 86 93 6B 07 2B 61 6C 8B 07 93 89 7A C9 96 90 1F 0C 66 DF 06 87 34 54 12 5B 2F 61 A9 80 75 1A D5 3C A7 A4 87 9A 40 3C 5E C3 74 20 38 E9 37 6D 59 F8 4C E6 49 6E 11 C2 5C 1A E8 DB CE 08 D3 72 AA A8 4A BD 3D E9 71 F4 28 25 1C 28 8B 4F 68 75 32 EF 57 5B 98 C6 60 B7 4D EC F1 D9 CA DC 30 C8 EC F7 44 80 31 61 B6 E9 F9 F9 73 BA DE 5F 52 41 AE 72 F8 ED 92 D2 43 92 BA 22 1D 85 3E 49 3D D3 DC 82 E3 19 4C 91 5E A1 88 41 67 AE B5 8B F5 78 F6 ED C8 8B 1A 7C 93 5F 91 C7 22 EF 69 00 1B E1 51 13 FC 35 21 9B 4A F7 88 4E E7 C0 2E 9F CB 9D AD C2 13 38 67 70 2A 66 A4 D7 13 A6 CD A8 ED 1A C6 54 73 56 0F 93 9A 17 CF 37 F8 64 B8 25 C1 E9 F8 13 85 43 80 49 C8 20 0D 6E 8C FD 66 7F
Exponent	65537
Key Size	2048 bits
Key Usage	Encrypt, Verify, Wrap, Derive
Signature	256 bytes : 77 1E A4 99 B4 1B D3 40 DE DF 74 38 95 0F 9F 76 1D 74 86 98 05 B6 CF 45 50 0D 11 94 57 BE 3D 3C F8 1B 0B 64 AA 93 61 68 AE E3 FC 0D E2 32 C7 3F D2 6D 47 73 23 A9 B7 51 83 06 47 4C 7E 8C 50 AB 7B FC C1 72 B0 D5 02 02 47 B7 F3 1B 27 23 5E 7E 9A 73 3B CD 5B 55 40 62 AA 62 EB 83 81 8A 27 8C 60 08 C0 0B 6A 64 33 4F 77 02 C4 E8 24 E5 85 69 AC E8 28 4A BF 69 D1 AD 50 E4 4F 16 1C DC 17 FD 72 40 1D A3 61 FE FB E0 34 0F 52 C0 12 E2 CD 70 0A 90 E8 67 C3 BC 35 F8 5A 2E 7C D7 42 F7 63 E0 03 35 85 47 8C AF 65 BD 8B A0 C9 F6 DC 0A 41 CF 88 DD 04 9B 0E 06 64 EC BA 0F 5E 1B EF 55 75 7A 4D 95 3E F5 D7 90 38 EA 9F 21 86 3D CC E7 B7 D3 52 E9 E6 35 DF BC 5D A2 76 A6 E2 B4 AD 3E 16 8F F2 09 6F 55 3A BA 31 A0 AA 50 93 8A 2E 10 A4 99 36 0D 1A D8 E6 9A 11 C2 F8 84 62 96 13 10 05 F4

OK

Certificates in SSL – website certificates

Class 3 Public Primary Certification Authority
↳ VeriSign Class 3 Public Primary Certification Authority – G5
↳ VeriSign Class 3 Secure Server CA – G3
↳ www.amazon.com

DNS Name	www.amzn.com
DNS Name	amzn.com
DNS Name	uedata.amazon.com
DNS Name	amazon.com
Extension	Certificate Policies (2 5 29 32)
Critical	NO
Policy ID #1	(2 16 840 1 113733 1 7 54)
Qualifier ID #1	Certification Practice Statement (1 3 6 1 5 5 7 2 1)
CPS URI	https://d.symcb.com/cps
Qualifier ID #2	User Notice (1 3 6 1 5 5 7 2 2)
User Notice	https://d.symcb.com/rpa
Extension	CRL Distribution Points (2 5 29 31)
Critical	NO
URI	http://sd.symcb.com/sd.crl
Extension	Certificate Authority Information Access (1 3 6 1 5 5 7 1 1)
Critical	NO
Method #1	Online Certificate Status Protocol (1 3 6 1 5 5 7 48 1)
URI	http://sd.symcd.com
Method #2	CA Issuers (1 3 6 1 5 5 7 48 2)
URI	http://sd.symcb.com/sd.crt
Fingerprints	
SHA1	71 B4 9F 93 60 A5 DB A1 15 32 01 35 86 82 CA F5 34 02 D2 A7
MD5	47 7E B3 A5 9F 0B A2 0B F5 4B C2 AC 7D 89 41 4F

OK

How to get an SSL certificate for a domain?

- Pay money
  - Verisign ($350)
  - Thawte ($150)
- Prove that your request comes from the legal holder of the domain
  - Letter from company
  - Notarized document
- CA may check
  - Existence of business
  - Ownership of the domain name
  - Employment status

SSL pitfalls

- Presence of SSL is indicated by a lock at the bottom of the browser window
- What does this mean?
  - Browser is engaging in a secure session with *some* server
  - Burden is on the user to check information about the certificate
  - There is no guarantee that the server that serves the certificate is the entity in the certificate
    - Users usually ignore browser warnings
- SSL is not a magical solution for Web security
  - Provides a secure pipe between user-server, but the user is responsible to verify identity of the server

SSL pitfalls

- Users don't check certificates
  - Users ignore warnings about invalid or expired certificates
  - Many users don't even know what certificates mean!
- Certificates can be easily obtained
- Browsers contain too many CAs
- Performance
  - Some sites turn it off during peak times

Too many CAs'!

Keychain Access

Click to unlock the System Roots keychain.

Keychains
- login
- Micr...ertificates
- Micr...ertificates
- System
- System Roots

Category
- All Items
- Passwords
- Secure Notes
- My Certificates
- Keys
- Certificates

TÜRKTRUST Elektronik Sertifika Hizmet Sağlayıcısı
Root certificate authority
Expires: Sunday, March 22, 2015 6:27:17 AM ET
This certificate is valid

Name	Kind	Date Modified	Expires	Keychain
Thawte Premium Server CA	certificate	--	Jan 1, 2021 6:59:59 PM	System Roots
thawte Primary Root CA	certificate	--	Jul 16, 2036 7:59:59 PM	System Roots
thawte Primary Root CA - G2	certificate	--	Jan 18, 2038 6:59:59 PM	System Roots
thawte Primary Root CA - G3	certificate	--	Dec 1, 2037 6:59:59 PM	System Roots
Thawte Server CA	certificate	--	Dec 31, 2020 6:59:59 PM	System Roots
Thawte Server CA	certificate	--	Jan 1, 2021 6:59:59 PM	System Roots
Thawte Timestamping CA	certificate	--	Jan 1, 2021 6:59:59 PM	System Roots
TRUST2408 OCES Primary CA	certificate	--	Dec 3, 2037 8:11:34 AM	System Roots
Trusted Certificate Services	certificate	--	Dec 31, 2028 6:59:59 PM	System Roots
Trustis FPS Root CA	certificate	--	Jan 21, 2024 6:36:54 AM	System Roots
TÜBİTAK UEK...layıcısı - Sürüm 3	certificate	--	Aug 21, 2017 7:37:07 AM	System Roots
TÜRKTRUST ...Hizmet Sağlayıcısı	certificate	--	Mar 22, 2015 6:27:17 AM	System Roots
TÜRKTRUST ...Hizmet Sağlayıcısı	certificate	--	Sep 16, 2015 6:07:57 AM	System Roots
TÜRKTRUST ...Hizmet Sağlayıcısı	certificate	--	Dec 22, 2017 1:37:19 PM	System Roots
TWCA Root Certification Authority	certificate	--	Dec 31, 2030 10:59:59 AM	System Roots
UCA Global Root	certificate	--	Dec 30, 2037 7:00:00 PM	System Roots
UCA Root	certificate	--	Dec 30, 2029 7:00:00 PM	System Roots
UTN - DATACorp SGC	certificate	--	Jun 24, 2019 3:06:30 PM	System Roots
UTN-USERFir...ication and Email	certificate	--	Jul 9, 2019 1:36:58 PM	System Roots
UTN-USERFirst-Hardware	certificate	--	Jul 9, 2019 2:19:22 PM	System Roots
UTN-USERFir...ork Applications	certificate	--	Jul 9, 2019 2:57:49 PM	System Roots
UTN-USERFirst-Object	certificate	--	Jul 9, 2019 2:40:36 PM	System Roots
VAS Latvijas Pasts SSI(RCA)	certificate	--	Sep 13, 2024 5:27:57 AM	System Roots
VeriSign Clas...n Authority - G3	certificate	--	Jul 16, 2036 7:59:59 PM	System Roots
VeriSign Clas...n Authority - G3	certificate	--	Jul 16, 2036 7:59:59 PM	System Roots
VeriSign Clas...n Authority - G3	certificate	--	Jul 16, 2036 7:59:59 PM	System Roots
VeriSign Clas...n Authority - G4	certificate	--	Jan 18, 2038 6:59:59 PM	System Roots
VeriSign Clas...n Authority - G5	certificate	--	Jul 16, 2036 7:59:59 PM	System Roots

Copy

200 items

Certificate Authorities: are they trustworthy?

- In general, yes, for all practical purposes
- However, they are not immune to attacks
  - In August 2011 a Dutch CA (DigiNotar) was hacked and and fraudulent digital certificates had been issued for *.google.com, *.microsoft.com, *.windowsupdate.com, www.update.microsoft.com, etc.
  - Man-in-the-middle attack becomes possible
    - Attacker sits in between client and server and can listen (and potentially tamper with) the communication
  - How was this addressed?
    - Web browsers needed to update their CA list (remove DigiNotar as a trusted CA)
    - Users can also manually remove a trusted CA from the browser

Certificate Authorities: are they trustworthy?

- http://nakedsecurity.sophos.com/2013/01/08/the-turktrust-ssl-certificate-fiasco-what-happened-and-what-happens-next/
- In mid 2011, TURKTRUST (a Turkish CA) introduced a flawed business process which made it possible for the company to generate and ship an intermediate certificate by mistake, when a regular certificate had been requested.
- TURKTRUST made such a blunder, and sent two intermediate certificates to organizations that had requested regular certificates.
- One of the certificates was revoked at the request of the customer who received it.
- The other incorrectly-issued intermediate certificate, issued to EGO, the public transport authority in Ankara, Turkey, remained valid.
- EGO now had the ability to sign SSL certificates for any domain name it chose, apparently with in the name of TURKTRUST.
- December 2012: on the EGO network, who was using Google's Chrome browser, received a warning about an unexpected certificate claiming to represent a google.com web property.

Certificate Authorities: are they trustworthy?

- A good survey of issues currently faced by CAs and the certification process:

  "SoK: SSL and HTTPS: Revisiting past challenges and evaluating certificate trust model enhancements", by J.Clark and P. C. van Oorschot, appeared in IEEE Security and Privacy Symposium 2013

  http://www.ieee-security.org/TC/SP2013/papers/4977a511.pdf

  - By and large, domain certificates are issued by CAs automatically
  - Other issues related to revocation, solutions to the revocation problem, etc.

Another interesting article

Security Collapse in the HTTPS Market

By A. Arnbak, H. Asghari, M. Van Eeten, and N. Van Eijk

In Communications of the ACM, October 2014

http://queue.acm.org/detail.cfm?id=2673311

- Analyzes the stakeholders in the HTTPS ecosystem
- Outlines systemic vulnerabilities of HTTPS
- Suggests regulatory and technological solutions
- Some of the findings:
  - Some CA's are "too big to fail"
  - Weakest Link: any CA can can sign certificates for any domain name

Certificate Transparency

- An alternative to Certificate Authorities?
- Certificate Transparency
  - Proposed by a team led by Ben Laurie (researcher at Google)
  - Store all certificates ever issued in a public, verifiable, append-only log
  - Clients refuse to honor certificates that are not present in at least a subset of trusted logs
  - In October 2016, Google announced that, starting in October 2017, all publicly trusted website certificates will have to comply with Chrome's Certificate Transparency policy to be trusted by its Chrome browser

Certificate Transparency

- Read more:
  - Original article published in the *ACM queue* magazine in September 2014: http://queue.acm.org/detail.cfm?id=2668154
  - https://blog.apnic.net/2017/06/12/certificate-transparency-rescue/
  - Official website:

    https://www.certificate-transparency.org/

Case Study: The “Heartbleed” attack

“Heartbleed”

- New vulnerability against OpenSSL-based software
- Discovered in April 2014
- Remained undiscovered for about 2 years!

How widespread is this? Quite a lot!

- Apache and nginx web servers use OpenSSL to secure communication (this accounts for over 66% of the Internet websites)
- OpenSSL is also used to protect:
  - Email servers (SMTP, POP, IMAP protocols)
  - Chat servers (XMPP protocol)
  - Virtual private networks (SSL VPNs)
  - Network appliances
- Vulnerable versions of OpenSSL (1.0.1 through 1.0.1f) have been rapidly adopted (they include TLS v. 1.1 and 1.2), as they provided protection against earlier attacks (e.g., BEAST)
- Ironically, smaller and more progressive services or those who have upgraded to latest and best encryption will be affected most.

What is the actual vulnerability?

- Quite simple! A missing bounds check in the code that handles TLS 'heartbeat' messages.
- An attacker can request that a server running TLS reveal a relatively large slice (up to 64KB) of its private memory space. This may include:
  - long-term server private keys
  - TLS session keys
  - confidential data like passwords
- The exploit leaves no trace!

- Note that this is not a flaw in the SSL/TLS protocol or the TLS heartbeat mechanism, but in the OpenSSL implementation of SSL/TLS

What is the actual vulnerability?

Untitled - Notepad

File Edit Format View Help

```
0700: BC 9C 2D 61 5F 32 36 30 35 26 2E 73 61 76 65 3D  ..-a_2605&.save=
0710: 26 70 61 73 73 77 64 5F 72 61 77 3D 06 14 CE 6F  &passwd_raw=...o
0720: A9 13 96 CA A1 35 1F 11 79 2B 20 BC 2E 75 3D 63  .....5..y+ ..u=c
0730: 6A 66 6A 6D 31 68 39 6B 37 6D 36 30 26 2E 76 3D  jfjm1h9k7m60&.v=
0740: 30 26 2E 63 68 61 6C 6C 65 6E 67 65 3D 67 7A 37  0&.challenge=gz7
0750: 6E 38 31 52 6C 52 4D 43 6A 49 47 4A 6F 71 62 33  n81RlRMCjIGJoqb3
0760: 75 69 72 61 2E 6D 6D 36 61 26 2E 79 70 6C 75 73  uira.mm6a&.yplus
0770: 3D 26 2E 65 6D 61 69 6C 43 6F 64 65 3D 26 70 6B  =&.emailCode=&pk
0780: 67 3D 26 73 74 65 70 69 64 3D 26 2E 65 76 3D 26  g=&stepid=&.ev=&
0790: 68 61 73 4D 73 67 72 3D 30 26 2E 63 68 6B 50 3D  hasMsgr=0&.chkP=
07a0: 59 26 2E 64 6F 6E 65 3D 68 74 74 70 25 33 41 25  Y&.done=http%3A%
07b0: 32 46 25 32 46 6D 61 69 6C 2E 79 61 68 6F 6F 2E  2F%2Fmail.yahoo.
07c0: 63 6F 6D 26 2E 70 64 3D 79 6D 5F 76 65 72 25 33  com&.pd=ym_ver%3
07d0: 44 30 25 32 36 63 25 33 44 25 32 36 69 76 74 25  D0%26c%3D%26ivt%
07e0: 33 44 25 32 36 73 67 25 33 44 26 2E 77 73 3D 31  3D%26sg%3D&.ws=1
07f0: 26 2E 63 70 3D 30 26 6E 72 3D 30 26 70 61 64 3D  &.cp=0&nr=0&pad=
0800: 36 26 61 61 64 3D 36 26 6C 6F 67 69 6E 3D 61 67  6&aad=6&login=ag
0810: 6E 65 73 61 64 75 62 6F 61 74 65 6E 67 25 34 30  nesaduboateng%40
0820: 79 61 68 6F 6F 2E 63 6F 6D 26 70 61 73 73 77 64  yahoo.com&passwd
0830: 3D 30 32 34                                      =024          &.pe
```

Alleged Yahoo user credentials visible due to Heartbleed (source: Mark Loman)

What is the actual vulnerability?

- The TLS Heartbeat mechanism is designed to keep connections alive even when no data is being transmitted
- Heartbeat messages sent by one peer contain random data and a payload length.
- The other peer is suppose to respond with a mirror of exactly the same data.

"When a HeartbeatRequest message is received and sending a HeartbeatResponse is not prohibited as described elsewhere in this document, the receiver MUST send a corresponding HeartbeatResponse message carrying an exact copy of the payload of the received HeartbeatRequest."

What is the actual vulnerability?

- The incoming (adversarially-generated) data contains a payload length ("payload") which is trusted without bounds checks.
- OpenSSL then allocates a buffer for its response, and copies "payload" data bytes from the pointer "pl" into it.
- There's no check to make sure that there are actually "payload" bytes in the incoming data, or that reading “payload” bytes from the “pl” pointer is in bounds.
- Hence the attacker gets a slice of data from main memory: one that's up to 64KB in length.

What is the fix for web servers?

- Patch OpenSSL by getting the latest version (at least version 1.0.1g), and recompile your applications; or patch your OS (Debian provides a patch)
- The fix of the code is just as simple as the bug was! Just add a bounds check!
- Or, just recompile your current version of OpenSSL with the -DOPENSSL_NO_HEARTBEATS option
- Wait… not so fast! Sadly, this is only the beginning ☹
- What if your private keys were already stolen?
- Since there's no way to tell whether a server has been exploited (and exploit code is now in the wild) you need to assume that it is.
- This means the safe move is to revoke your certificate and get a new one. Have fun ☹

On the importance of PFS

- If the handshake in SSL/TLS uses a protocol that provides perfect forward secrecy (PFS) for the session key establishment, then the Heartbleed attack will not lead to the compromise of past sessions (traffic intercepted by the attacker in the past will not be vulnerable to decryption)
- It is possible to run TLS with PFS, although it is not very commonly used and will incur some performance degradation
- A quick search reveals that in 2011 Google announced support for PFS on its HTTPS websites:

http://googleonlinesecurity.blogspot.com/2011/11/protecting-data-for-long-term-with.html

What should you do?

- The safest thing would be for users to generate new passwords for their most critical sites (such as email, banking, and social networks) if those sites utilize Apache, Nginx or show as vulnerable to the Heartbleed bug.

- However, users should wait until their sites have replaced their certificates, with a start date after April 8th, 2014.

What is the current progress of patching?

- As of April 9, 2014:

- Yahoo reported that its “team has successfully made the appropriate corrections across the main Yahoo properties (Yahoo Homepage, Yahoo Search, Yahoo Mail, Yahoo Finance, Yahoo Sports, Yahoo Food, Yahoo Tech, Flickr and Tumblr) and we are working to implement the fix across the rest of our sites right now.”

Where can you read more about Heartbleed?

- Sadly, this new attack follows a series of attacks against TLS, including BEAST, CRIME, Lucky13, gotofail;, GnuTLS
- Read more at:
  - http://heartbleed.com/
  - https://blog.cryptographyengineering.com/2014/04/attack-of-the-week-openssl-heartbleed.html
  - http://blog.existentialize.com/diagnosis-of-the-openssl-heartbleed-bug.html
  - http://www.cnet.com/news/how-to-protect-yourself-from-the-heartbleed-bug/
- A history of SSL/TLS attacks:
  - http://www.nds.rub.de/media/nds/veroeffentlichungen/2013/08/19/paper.pdf

What to read?

- HTTP and HTML: Chapter 7.1.1 from the Textbook
- PKI: Chapter 1.3.5 from the Textbook
- SSL: Chapter 7.1.2 from the Textbook